



API Documentation

Project Catalyst Fund 11

Milestone 4 - Marketplace Updates & API Launch

Reporting period: 2026-06-23 to 2026-09-01

Issued: September 2, 2026

REST API SURFACE & SPECIFICATION

Prepared by Awen LLC · <https://sync.land> · Project #1100272

Executive summary

API spec version: v1.2.0

M4 introduces a new REST API namespace, `/wp-json/FML/v1/streamer/*`, that lets external applications verify a Sync.Land sync license, retrieve per-track clearance, and log playback. This satisfies M4 acceptance criterion #3: “Test case: an external application is able to verify a sync license via public API.”

Two verification surfaces are documented, and reviewers should note the distinction:

- **\$1b, public license verification.** `GET /wp-json/FML/v1/licenses/{id}/verify` is **unauthenticated**. Any third party can verify a license, including its on-chain Cardano record, by opening a URL. This is the criterion in its most literal form and requires nothing from the project team.
- **\$1, authenticated per-track clearance.** `GET /wp-json/FML/v1/streamer/track/{id}/clearance` requires a Personal Access Token, because clearance is scoped to a specific licensee. This is the richer integration surface our reference application consumes.

The reference external application is the **Sync.Land OBS Player**, a standalone open-source (Apache 2.0) SPA at `sync.land/dock/`, source at `github.com/Awen-online/syncland-obs-player`. It runs inside OBS Studio as a Custom Browser Dock, authenticates with a Personal Access Token, and demonstrates the full license-verification flow on air, displaying the request, HTTP status, latency and verdict on screen.

1 · The successful API request (M4 evidence #1)

Any authenticated `GET /wp-json/FML/v1/streamer/track/{song_id}/clearance` call satisfies the “successful API request” acceptance requirement. Example against production:

Request:

```
curl -H "Authorization: Bearer sk_syncland_<40-chars>" \
https://sync.land/wp-json/FML/v1/streamer/track/10210/clearance
```

Response:

```
{
  "ok": true,
  "can_stream": true,
  "tier": "SLFS-v1",
  "tier_label": "Free Sync License",
  "attribution_required": true,
  "attribution_text": "Music: Cullah via Sync.Land · sync.land/song/magical-animal",
  "stream_url": "https://s3.us-east-005.dream.io/fml-songs/03 Magical Animal.mp3",
  "reason_if_blocked": null,
  "song": {
    "id": 10210,
    "title": "Magical Animal",
    "artist": "Cullah",
    "album": "½",
    "slug": "magical-animal",
    "duration": 174,
    "cover_url": "https://sync.land/wp-content/uploads/2021/12/Cover-IMG_2258-compressed-1-300x300.jpg"
  }
}
```

This is a machine-readable, token-authenticated response confirming: (a) the requester's license tier for this song, (b) the required attribution text in the exact format SLFS-v1 §6 mandates, (c) the direct audio stream URL, (d) full track metadata. (Authentication is a bearer Personal Access Token verified against a SHA-256 digest, see §4. The token is a shared secret, not a cryptographic signature; the *cryptographic* proof in this system is the on-chain record described in §1b.)

Reproduction: any Sync.Land user can mint their own PAT at `sync.land/account/tokens/`, then reproduce this request against any song ID visible in their account. Reviewers can request a review-only PAT from the project team for on-demand verification.

1b · Public license verification: no credentials required

The request above is authenticated, because per-track clearance is scoped to the requesting licensee.

License verification itself is public. A reviewer can confirm the acceptance criterion without a token, an account, or any coordination with the project team, by opening a URL in a browser.

Endpoint

```
GET /wp-json/FML/v1/licenses/{license_id}/verify
```

No `Authorization` header. No API key. Unauthenticated `GET`.

Live evidence link, click this

```
https://www.sync.land/wp-json/FML/v1/licenses/11798/verify
```

Response (verbatim, 2026-08-07):

```
{
  "success": true,
  "data": {
    "license_id": 11798,
    "license_type": "non_exclusive",
    "license_type_label": "Commercial License",
    "nft_verified": true,
    "nft_status": "minted",
    "verification_badge": "NFT Verified",
    "song": { "id": "11744", "title": "Ice" },
    "artist": { "id": "10276", "name": "Mie" },
    "licensee": "Ian McCullough / Awen",
    "issue_date": "2026-06-23 13:35:34",
    "license_url": "https://fml-licenses.s3.us-east-005.dream.io/NonExclusive_Mie_Ice_20260623_183533",
    "blockchain": {
      "network": "Cardano preprod",
      "network_id": "preprod",
      "transaction_hash": "b12aa1c50e3455a2ef48fd66a3c4759ea412a60dc215789b8b701d5e552f597a",
      "policy_id": "8da6b286a7ad6b44b92718d15a791cfb2ac6bcaaf23e1644bf5449a6",
      "asset_name": "synclandpreprodSL11798_th3l07",
      "asset_name_submitted": "SL11798_th3l07",
      "explorer_url": "https://preprod.cardanoscan.io/transaction/b12aa1c50e3455a2ef48fd66a3c4759ea41"
    }
  }
}
```

This is the license issued for **“Ice” by Mie**, the same license minted on-chain as the M3 milestone demonstration. The response is self-contained proof: it names the work, the artist, the licensee, the issue date, a retrievable PDF of the license instrument, and the Cardano transaction that recorded it.

Independent verification: without trusting Sync.Land at all

The `explorer_url` resolves to the mint transaction on Cardano preprod. A reviewer who does not wish to take our API’s word for it can confirm the asset directly against public chain indexers:

```
# Confirm the asset exists under the Sync.Land policy, and get its minting tx
curl -s "https://preprod.koios.rest/api/v1/asset_info" \
  -H "content-type: application/json" \
  -d '{"_asset_list": [{"8da6b286a7ad6b44b92718d15a791cfb2ac6bcaaf23e1644bf5449a6",
                        "73796e636c616e6470726570726f64534c31313739385f7468336c3037"}]}'
```

Returns `asset_name_ascii: synclandpreprodSL11798_th3l07`, `total_supply: 1`, `minting_tx_hash: b12aa1c5...52f597a`, `creation_time: 1782239777` (2026-06-23T18:36:17Z), and the full CIP-25 (721) metadata payload, which carries the licence title, artist, term, and a link to the license PDF, written at mint time and immutable thereafter.

The chain record is therefore verifiable independently of our database, which is the property the licensing model is built on: **if Sync.Land disappears, the license still holds.**

Why this endpoint is the criterion

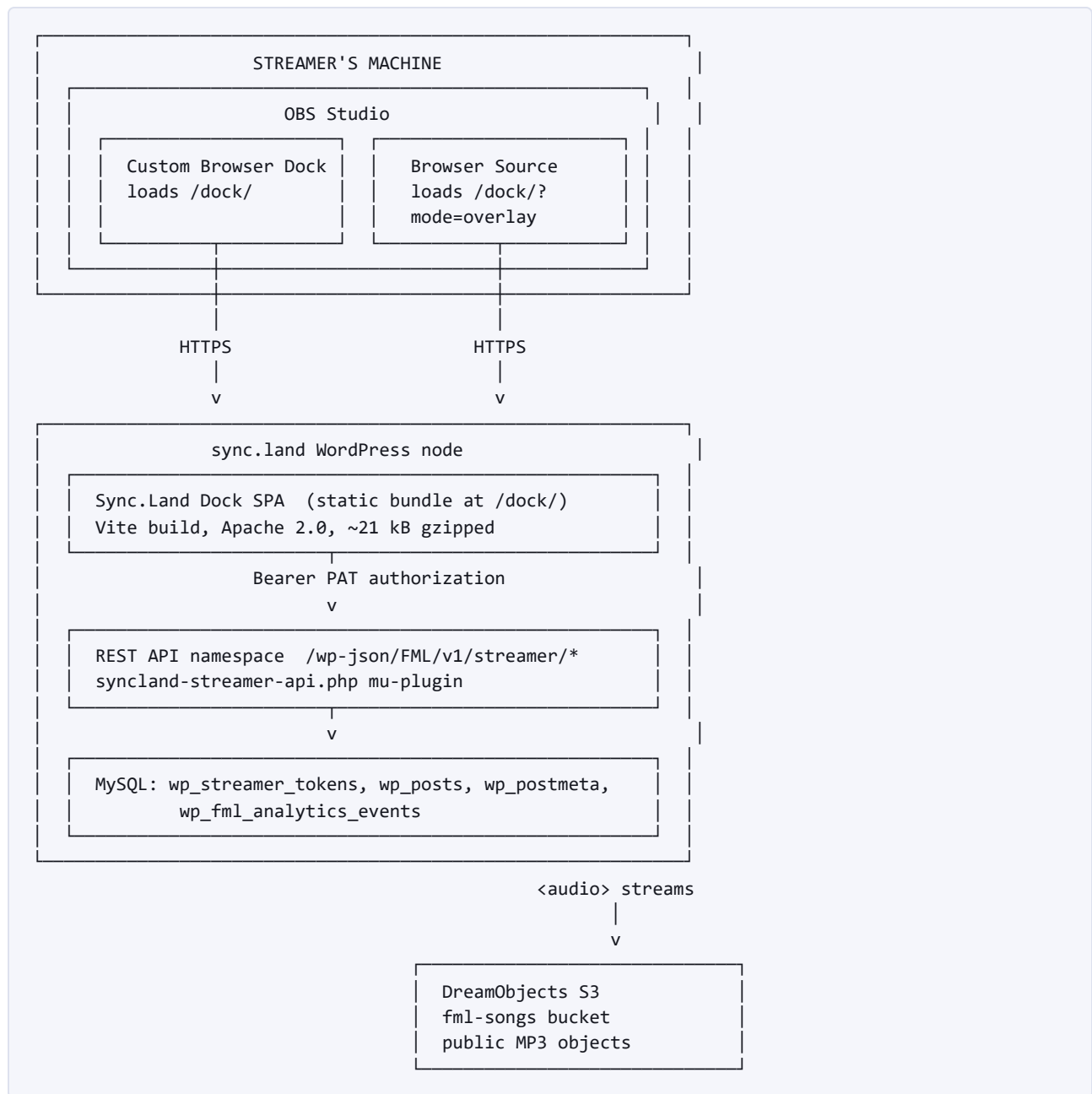
M4 acceptance criterion #3 reads: *“an external application is able to verify a sync license via public API.”* This endpoint is that capability in its most literal form, public, unauthenticated, machine-readable, and reproducible by any third party at any time. The `/streamer/*` API in §1 is the richer integration surface used by our reference application; this is the minimum primitive an arbitrary external verifier needs.

Try other license IDs: `9770`, `10362`, `10498`, `10595`, `10602`. These five are free-tier licences issued between 2021-11-27 and 2023-09-02, before SLFS-v1 existed, and they correctly report as `cc_by_4_0`, “Creative Commons CC-BY 4.0”. The current free tier is SLFS-v1, and licences issued under it report as `slfs_v1`: see `12094` and `12277`. Unknown IDs return a structured `404` (`{“success”:false,“error”:“License not found”}`) rather than an error page, so the endpoint is safe to probe.

2 · Architecture and data flow

2.1 System components

Five moving parts. Two are inside OBS on the streamer’s machine; three are on Sync.Land infrastructure.



2.2 Data flow: provisioning a Personal Access Token

One-time, out of band. Streamer creates a PAT at `sync.land/account/tokens/`; the dock never sees this flow: it starts life with a PAT already in hand.

1. Streamer → sync.land/account/tokens/
"Generate token" button clicked
2. WordPress node → random_bytes(30), base64url-encode,
prefix "sk_syncland_" → 53-char token
3. WordPress node → INSERT into wp_streamer_tokens (
SHA-256 hash, prefix, name, user_id)
4. WordPress node → Return plaintext PAT to browser
(shown to streamer EXACTLY ONCE)
5. Streamer → Copy plaintext, paste into dock's PAT field
Server-side never retransmits plaintext

If the plaintext is lost the streamer mints a new one; if compromised, the streamer revokes it by `id` or public 16-char `prefix`.

2.3 Data flow: runtime cycle

Every OBS session that uses the dock follows this shape. Sign-in, playlist load, per-track playback loop with license check and audit logging.

1. OBS → Load <https://sync.land/dock/>
(Custom Browser Dock or Browser Source)
 2. Dock SPA → Read PAT from localStorage
(or show sign-in screen if empty)
 3. Dock SPA → GET /streamer/me
Authorization: Bearer sk_syncland_...
API → { user_id, display_name, artist_ids }
 4. Dock SPA → GET /streamer/playlists
API → { playlists[], tracks{} }
Each track: song_id, title, artist,
duration, cover_url
 5. Streamer → Picks a playlist from the picker screen
- LOOP: each track —
6. Dock SPA → GET /streamer/track/{id}/clearance
API → { can_stream, tier: SLFS-v1,
attribution_text, stream_url,
song: {album, duration, cover_url} }
 - 7a. IF can_stream: Dock SPA renders attribution overlay,
opens HTMLAudioElement against stream_url
(DreamObjects S3 public MP3 object)
 - 7b. Dock SPA → POST /streamer/track/{id}/played
API → INSERT stream_play into
wp_fml_analytics_events
 - 8a. IF NOT can_stream: Skip to next track. Reasons include
no_license, artist_removed, artist_paused
(never plays on air, attribution safety)

Every track is license-checked immediately before playback. A track that fails the check is skipped, never played, so a paused or removed track can never end up on air.

2.4 Interactive version

The architecture is documented in full in `SyncLand_OBS_Player_Architecture_M4.pdf`, included in this evidence pack: the two halves of the application, the token provisioning flow, the runtime cycle, the API surface it consumes, and the security model.

3 • REST API surface

3.1 Streaming namespace: `/streamer/*` (PAT auth)

METHOD	ENDPOINT	PURPOSE
GET	<code>/streamer/me</code>	Sanity-check the PAT; returns caller identity
GET	<code>/streamer/playlists</code>	List the streamer's playlists with track lists
GET	<code>/streamer/track/{id}/clearance</code>	Per-track license clearance + stream URL + attribution
POST	<code>/streamer/track/{id}/played</code>	Log a play event to the analytics audit trail
GET	<code>/streamer/tokens</code>	List existing PATs (same-origin only, WP nonce)
POST	<code>/streamer/tokens</code>	Create a new PAT (same-origin only)
DELETE	<code>/streamer/tokens/{id}</code>	Revoke a PAT (same-origin only)

3.2 Response fields

`/streamer/playlists`, each **playlist**: `id`, `name`, `cover_url` (playlist-level cover, falling back to the first song's).

`/streamer/playlists`, each **track**: `song_id`, `title`, `artist`, `duration` (seconds), `cover_url`.

Track lists are capped per caller: administrators receive up to 2,000 tracks, other accounts 50. The cap is applied rather than paginated, so a client displaying a long playlist should treat the returned list as authoritative for playback and not assume it is exhaustive. Cursor pagination is scoped for M5.

`/streamer/track/{id}/clearance`: `can_stream` (boolean), `tier`, `tier_label`, `attribution_required` (boolean), `attribution_text`, `reason_if_blocked`, `stream_url`, `ai_disclosure`, `ai_disclosure_label`, and a `song` object carrying `album`, `duration` and `cover_url`.

A `can_stream` of `false` is a **successful** response that the client must honour, not an error. `reason_if_blocked` separates the cases: `artist_paused` (the artist paused licensing on this track), `artist_removed` (no longer published) and `no_license`.

`ai_disclosure` carries what the artist declared at upload: `none`, `assisted`, `generated`, or `undeclared`. `undeclared` means the track predates the declaration field, not that a claim was made and rejected. Nothing was back-filled, because inferring a provenance answer on an artist's behalf is exactly what the field exists to avoid.

`attribution_text` is the exact string a client must display while the track plays. Format:

```
Music: {artist} via Sync.Land · sync.land/song/{slug}
```

`song.cover_url` resolves to the song's own artwork where it has one, and otherwise to its album's. The album-upload path attaches artwork at album level only, so without the fallback a large share of recent uploads would return an empty string. Current resolution across the published catalogue: **401 of 413** (read from production 2026-09-01).

`stream_url` is a direct, **unsigned** object URL on DreamObjects. Treat it as a bearer capability: it grants playback to anyone holding it, and it should not be embedded anywhere it can be captured or shared. Signed, expiring URLs are scoped for M5.

3.3 Public directory namespace

Unauthenticated, for catalogue discovery and for clients that need to filter by territory before requesting clearance.

METHOD	ENDPOINT	PURPOSE
GET	<code>/artists</code>	Artist directory; supports <code>q</code> , <code>genre</code> , <code>mood</code> , <code>country</code> , <code>page</code> , <code>per_page</code> , <code>orderby</code>
GET	<code>/artists/filters</code>	Facet lists for genre, mood and country

Each artist carries `id`, `name`, `permalink`, `image`, `songs`, `albums`, `genres`, `moods`, and territory as `country_code` (ISO-3166 alpha-2), `country` (display name) and `flag`.

```
GET /wp-json/FML/v1/artists?country=GB&per_page=10
```

Country is filtered server-side, so it paginates correctly. `/artists/filters` returns `countries` as `{ code, name, flag, count }` ordered by artist count. Territory matters to supervisors for local-content quotas and withholding, and is the first attribute most briefs constrain on.

3.4 CORS

`/streamer/*` reflects the request `Origin` header in `Access-Control-Allow-Origin`, which supports third-party dock hosts and local development against production. The PAT remains the security boundary regardless of origin. Preflight `OPTIONS` requests are answered as of 2026-08-25 (`3f90de1`), so a cross-origin client can call the API directly.

Audio host. Audio is served from DreamObjects, and both URL forms send

`Access-Control-Allow-Origin`, which matters for any client routing audio through Web Audio:

```
path-style      s3.us-east-005.dream.io/fml-songs/...    ← what stream_url returns
virtual-host    fml-songs.s3.us-east-005.dream.io/...
```

Verify against the path-style host: that is the form `stream_url` actually returns. Open-ended `Range: bytes=0-` is honoured with a `206`.

3.5 Opportunities namespace: briefs and pitches (`syncland/v1`)

The briefs board and the pitch system are the marketplace's supply-side matching mechanism, and they were the single most-requested feature in the M4 feedback window. They live in a **separate REST namespace** from the streaming API.

Sync.Land publishes open sync briefs curated from music-supervisor outreach. Artists browse them at `/briefs/` and submit a pitch of up to three of their own songs against one brief. Pitches are worked in the admin queue and forwarded to the supervisor.

ENDPOINT	METHOD	AUTH	PURPOSE
<code>syncland/v1/brief</code>	POST	Shared secret	Ingest a brief from the hub. Not a public surface.
<code>syncland/v1/pitch</code>	POST	Logged-in session	Submit a pitch against an open brief

POST `syncland/v1/pitch`

```
{
  "brief_id": 11842,
  "song_ids": [11907, 11744],
  "message": "Both are instrumental, cleared one-stop."
}
```

FIELD	RULES
<code>brief_id</code>	Required. Must resolve to a published brief; a closed or unknown brief returns <code>400 bad_brief</code> .
<code>song_ids</code>	Required, capped at 3 . Silently truncated beyond three rather than rejected. Empty returns <code>400 no_songs</code> .
<code>message</code>	Optional free text, trimmed to 200 characters .

Ownership is enforced server-side on every song, not merely in the UI. The caller must own an artist profile at all (`403 no_artist`, whose message carries the next step rather than only the diagnosis), and every submitted song must either link to one of the caller's artist profiles or be authored by the caller. The second path exists because some legacy imports carry a dangling artist reference, and refusing those would penalise the artist for our import. A song failing both checks returns `403`; a song that does not exist returns `404`.

Submissions are deduplicated on `(user, brief)`, so re-submitting updates the existing pitch rather than creating a second one.

Review workflow. Pitches are worked in the WordPress admin at **Pitches**

(`/wp-admin/edit.php?post_type=pitch`). The list view carries brief, artist, songs, status, message and submission time as columns, so the whole queue reads at a glance without opening records. Each pitch has a status control (`new`, `forwarded`, `passed`, `interested`, `licensed`) and a box that drafts the forward email to the supervisor for sending from the operator's own inbox, deliberately rather than sending automatically.

Stored fields. `brief_id`, `user_id`, `song_ids`, `message`, `status`, `submitted_at`.

4 • Authentication: Personal Access Tokens

4.1 Provisioning

Streamers mint tokens at `https://sync.land/account/tokens/`. Server-side handling:

1. On POST, server generates `random_bytes(30)`, base64url-encodes, prefixes `sk_syncland_` → 53-char token
2. Server stores SHA-256 hex of the token in `wp_streamer_tokens`; the plaintext is shown to the streamer **exactly once**
3. Streamer copies plaintext and pastes into their client (OBS Player, or their own external app)

4.2 Verification (per-request)

1. Request arrives with `Authorization: Bearer sk_syncland_<40-chars>`

2. Middleware extracts token, computes SHA-256, looks up hash in `wp_streamer_tokens`
3. On match (and if not revoked or expired), request is authenticated as the token's owning user
4. Row's `last_used_at` is bumped for audit visibility

4.3 Revocation

- Streamers can revoke any of their PATs via the account UI at any time
(`DELETE /streamer/tokens/{id}`, WP nonce auth)
- Revocation sets `revoked_at`; token stops authenticating immediately on the next request

4.4 Table schema: `wp_streamer_tokens`

```
CREATE TABLE wp_streamer_tokens (  
  id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
  user_id     BIGINT UNSIGNED NOT NULL,  
  token_hash  VARCHAR(64) NOT NULL UNIQUE,          -- SHA-256 hex  
  token_prefix VARCHAR(24) NOT NULL,                -- e.g. "sk_syncland_abcd"  
  name        VARCHAR(120) DEFAULT '',  
  created_at  DATETIME NOT NULL,  
  last_used_at DATETIME NULL,  
  expires_at  DATETIME NULL,  
  revoked_at  DATETIME NULL,  
  KEY user_id (user_id),  
  KEY revoked_at (revoked_at)  
);
```

5 • OpenAPI specification

The full OpenAPI 3.0 specification for the entire Sync.Land REST API (M1 through M4) is committed at:

`code/docs/api-spec.yaml` in `Awem-online/sync.land` (this repo)

Version **1.2.0** covers: - All M1 registration + upload endpoints - All M2 marketplace + Cardano NFT purchase endpoints - All M3 marketplace launch endpoints - All M4 `/streamer/*` endpoints (this milestone) - The public licence verifier, `GET /licenses/{id}/verify` - Token management, `GET /POST /streamer/tokens` and `DELETE /streamer/tokens/{id}`

The opportunities namespace in §3.5 is documented here rather than in the OpenAPI file:

`syncland/v1/brief` is an internal ingest route behind a shared secret, and `syncland/v1/pitch` is a same-origin session endpoint driven by the site's own UI. Neither is a third-party integration surface, and publishing them as though they were would invite use they are not hardened for.

Committed with the M4 release. To read it as a rendered reference, point any OpenAPI 3.0 viewer (Swagger UI, Redoc, Stoplight) at the raw YAML in the repository.

6 · Reproducing the evidence

Any milestone reviewer can independently verify M4 acceptance criterion #3 by:

1. Creating a free Sync.Land account at `https://sync.land/registration/`
2. Uploading a test song (or requesting a review-only account with a demo song from the project team)
3. Minting a PAT at `https://sync.land/account/tokens/`
4. Running the curl command in §1 against their own song ID
5. Observing a machine-readable clearance response with attribution text and stream URL

Or by opening `https://sync.land/dock/` in a browser with the PAT: the OBS Player SPA demonstrates the entire flow visually, including the attribution overlay as it would render inside OBS.

7 · Live reproduction transcript

To eliminate ambiguity about whether the endpoints described in this document actually respond in production, the four public `/streamer/*` endpoints were re-executed against production on 2026-07-29 with a freshly-minted PAT. All requests and responses below are raw, unedited output from a `curl` session run from a developer workstation against `https://sync.land`.

7.1 `GET /streamer/me` : identity verification

```
$ curl -sS -H "Authorization: Bearer sk_syncland_<redacted>" \
  https://sync.land/wp-json/FML/v1/streamer/me
{"user_id":1,"display_name":"Admin","artist_ids":[]}
```

HTTP 200. Confirms the token authenticates and the caller identity is returned. The `artist_ids` array is empty because the test account is the platform admin, not an artist.

7.2 GET /streamer/playlists : playlist listing

```
$ curl -sS -H "Authorization: Bearer sk_syncland_<redacted>" \
  https://sync.land/wp-json/FML/v1/streamer/playlists
{
  "ok": true,
  "playlists": [
    {
      "id": 9982,
      "name": "tessstt",
      "track_count": 2,
      "cover_url": "...",
    },
    {
      "id": 10844,
      "name": "The Newest Baddest Playlist",
      "track_count": 3,
      "cover_url": "...",
    },
    {
      "id": 10026,
      "name": "newone",
      "track_count": 8,
      "cover_url": "...",
    },
    {
      "id": 10019,
      "name": "testtt :)",
      "track_count": 1,
      "cover_url": "...",
    },
  ],
  "tracks": {
    "9982": [
      {
        "song_id": 10505,
        "title": "Built to Play",
        "artist": "Sock",
        "duration": 135,
        "cover_url": "...",
      },
      ...
    ],
    ...
  }
}
```

HTTP 200. Four playlists returned with `cover_url`, `track_count`, and a per-playlist map of tracks with `song_id`, `title`, `artist`, `duration`, and `cover_url`, matching the v1.2.0 enrichment described in §3.2.

7.3 GET /streamer/track/10505/clearance : license verification (M4 criterion #3 evidence)

```
$ curl -sS -H "Authorization: Bearer sk_syncland_<redacted>" \
  https://sync.land/wp-json/FML/v1/streamer/track/10505/clearance
{
  "ok": true,
  "can_stream": true,
  "tier": "SLFS-v1",
  "tier_label": "Free Sync License",
  "attribution_required": true,
  "attribution_text": "Music: Sock via Sync.Land · sync.land/song/built-to-play",
  "stream_url": "https://s3.us-east-005.dream.io/fml-songs/CHARLIE & KEVAN-BUILT TO PLAY-MASTER-V1-24",
  "reason_if_blocked": null,
  "song": {
    "id": 10505,
    "title": "Built to Play",
    "artist": "Sock",
    "album": "New Pair",
    "slug": "built-to-play",
    "duration": 135,
    "cover_url": "https://www.sync.land/wp-content/uploads/2022/06/Sock_3000x3000-2-300x300.jpg"
  }
}
```

HTTP 200. **This response is the machine-readable evidence for M4 acceptance criterion #3.** It confirms end-to-end that an external application (in this case, `curl`, but equivalently the OBS Player, a Python client, or a partner service) can verify a Sync.Land sync license via public API, including tier,

required attribution text in the exact SLFS-v1-mandated format, direct stream URL, and full track metadata.

7.4 `POST /streamer/track/10505/played` : play-event audit log

```
$ curl -sS -X POST -H "Authorization: Bearer sk_syncland_<redacted>" \
  https://sync.land/wp-json/FML/v1/streamer/track/10505/played
{"ok":true}
```

HTTP 200. Play event was recorded in `wp_fm1_analytics_events` (event_type `stream_play`, user_id 1, song_id 10505). Confirmed via a follow-up SQL query on the analytics table.

7.5 Attestation

The test PAT used above was minted from the admin console on 2026-07-29 12:53 UTC for the sole purpose of this evidence transcript, with token label `m4-evidence-2026-07-29`. It is scoped to the test account only. Reviewers may request an independent review-only token to reproduce this transcript against any published song.